OUTPUT ANALYSIS

Smart Hospital Management System Using Doubly Linked List and FIFO Queue in C++

1. Time Complexity Analysis

Let n be the total number of patient records and w be the number of patients currently waiting (where w <= n).

Operation	Time Complexity	Reason
Register Patient	**O(w) worst case**	List and queue insertion are O(1), but calculating the queue position traverses the waiting queue.
Call Next Patient	**O(1)**	Removes the front queue node and updates front/rear pointers in constant time.
Search Patient	**O(n)**	May traverse the full doubly linked list to find a token or determine that it is absent.
Display All Records	**O(n)**	Visits each patient record once from head to tail.
Display Records in Reverse	**O(n)**	Visits each patient record once from tail to head.
Count Waiting Patients	**O(w)**	Traverses each node in the waiting queue once.
Hospital Dashboard	**O(n)**	Traverses all patient records to calculate status totals.
Discharge Patient	**O(n)**	Searching for the patient is O(n); updating the status after locating the node is O(1).

2. Space Complexity Analysis

Measure	Complexity	Explanation
Total Space	**O(n)**	Each patient uses one dynamically allocated node containing patient data and record/queue pointers. Storage grows with the number of patients.
Auxiliary Space per Operation	**O(1)**	Search, display, queue processing, counting, dashboard, and discharge use a fixed number of temporary pointers and counters. Registration allocates one node, which adds O(1) space for that new patient.

Interpretation: Keeping head/tail and front/rear pointers makes patient insertion into the record list and FIFO queue, and removal from the queue front, constant-time. Operations that inspect records or calculate a count remain linear.